

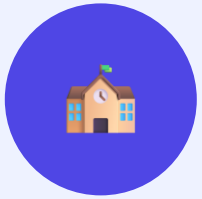
Sistema de Moeda Estudantil

Laboratório de Desenvolvimento de Software

Profa. Milena Menezes Adão

Lucas Lima Magalhães Lafetá Chaves

Como o sistema funciona



Professor

Recebe 1.000 moedas por semestre e reconhece alunos com moedas e uma mensagem.

envia moedas



Aluno

Acumula as moedas, consulta o extrato e troca por vantagens das empresas.

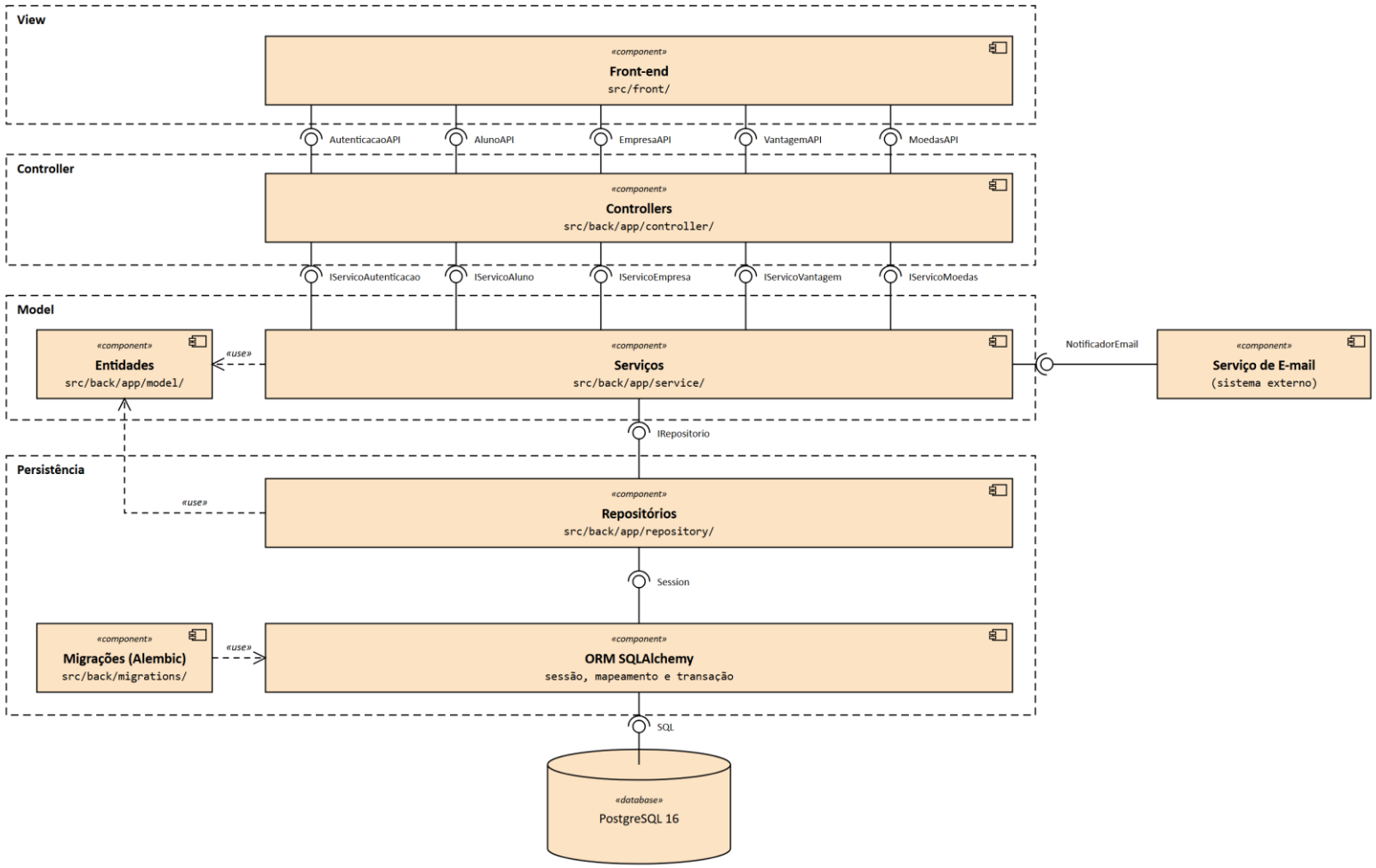
resgata vantagem



Empresa parceira

Cadastra vantagens com foto e custo em moedas e confere o cupom do aluno.

Arquitetura em camadas (MVC)



View

React + TypeScript consome a API por HTTP

Controller

FastAPI: rotas, validação e perfis

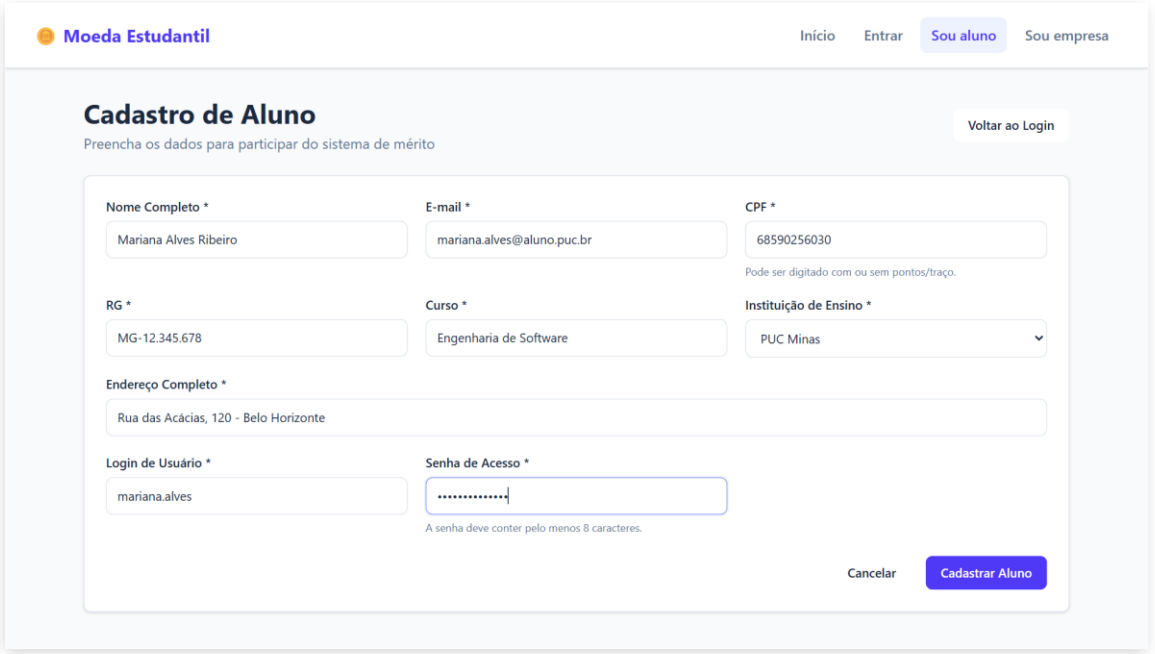
Model

Serviços de regra de negócio e entidades

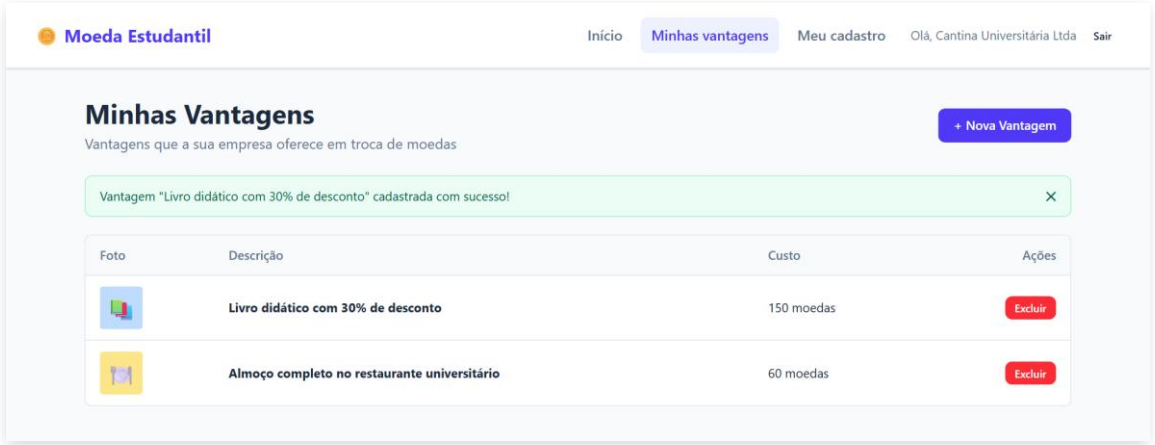
Persistência

Repositórios, SQLAlchemy e PostgreSQL

Demonstração: cadastro, login e vantagens



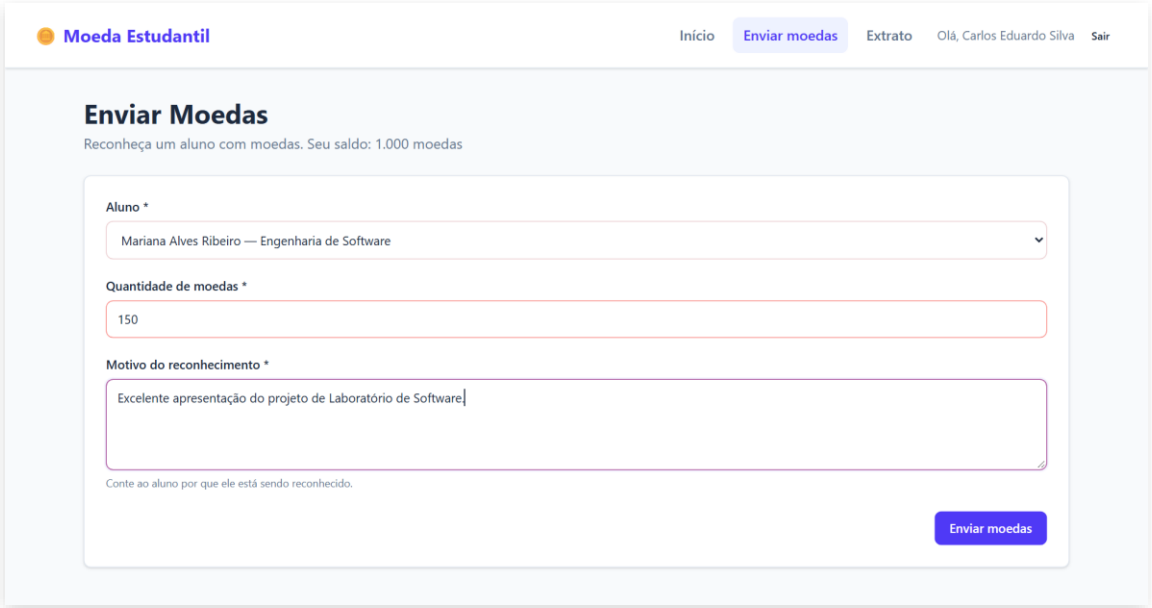
Cadastro do aluno, com erros por campo



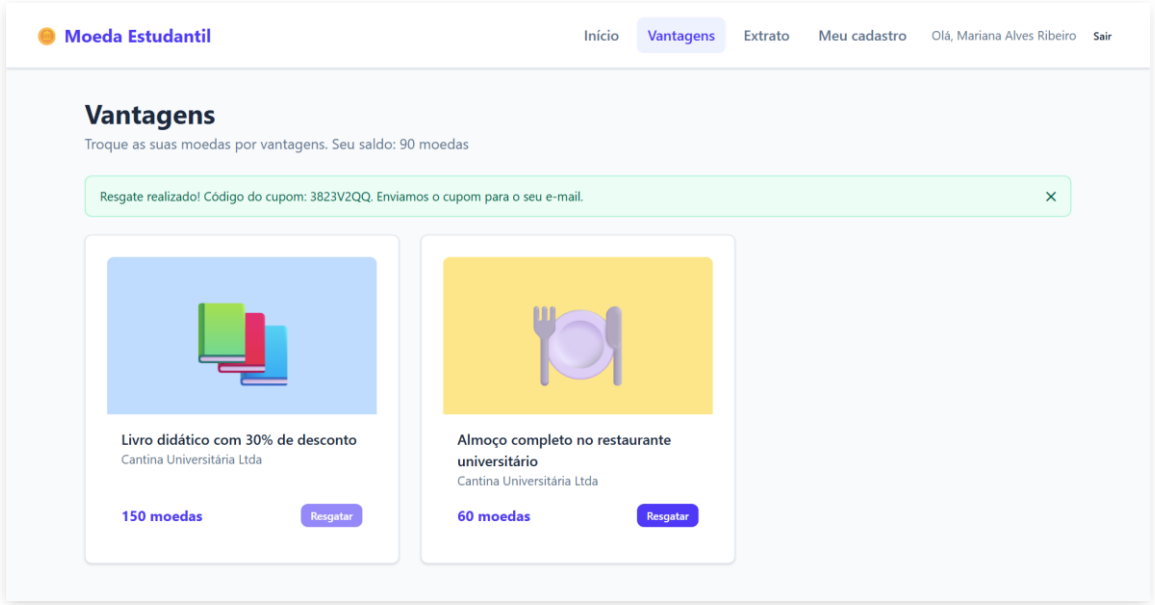
A empresa cadastra vantagens com foto

Cada perfil só enxerga as próprias telas: a empresa não acessa o extrato e o aluno não acessa o envio de moedas.

Demonstração: envio de moedas e resgate



Professor: aluno, valor e motivo obrigatório



Aluno: resgate com o código do cupom

O saldo é conferido antes de cada envio e de cada resgate, e o botão de resgatar fica desabilitado sem saldo.

Extrato e e-mails



Extrato do aluno: o que entrou e o que saiu

- No resgate saem dois e-mails: o cupom para o aluno e o aviso para a empresa.
- O código é o mesmo nos dois de propósito: o aluno apresenta o cupom e a empresa confere com o código que recebeu.



E-mails recebidos no Gmail durante o teste

Oficina: as tecnologias por camada

View	**React · TypeScript · Tailwind**	A tela no navegador: componentes, tipos e estilo.
Controller	**FastAPI · Pydantic · JWT · bcrypt**	Recebe o pedido, valida os dados e protege o acesso.
Model	**Python (serviços e entidades)**	As regras do negócio: saldo, envio, resgate e cupom.
Persistência	**SQLAlchemy · PostgreSQL · Alembic**	Repositórios gravam e leem do banco; o esquema é versionado.
Ambiente	**Docker Compose**	Sobe banco, API e tela juntos, com um comando.

React e TypeScript: a tela como componente

Camada View

O que é

O React monta a interface com componentes: funções que devolvem um pedaço de tela. O TypeScript acrescenta tipos ao JavaScript e avisa dos erros antes de rodar.

Como usar

1. Criar o projeto: npm create vite@latest oficina -- --template react-ts
2. Instalar: cd oficina && npm install
3. Escrever o componente em src/Livros.tsx
4. Rodar: npm run dev e abrir localhost:5173

src/Livros.tsx

```
type Livro = { id: number; titulo: string };

export function Livros() {
  const [livros, setLivros] = useState<Livro[]>([]);

  useEffect(() => {
    fetch("/api/livros")
      .then((r) => r.json())
      .then(setLivros);
  }, []);

  return (
    <ul>
      {livros.map((l) => <li key={l.id}>{l.titulo}</li>)}
    </ul>
  );
}
```

Tailwind CSS: estilo por classes

Camada View

O que é

O Tailwind é um CSS por utilitários: cada classe faz uma coisa pequena e você combina várias direto no elemento, sem escrever um arquivo de CSS à parte.

Como usar

1. Instalar: npm install tailwindcss @tailwindcss/vite
2. Registrar o plugin no vite.config.ts
3. Importar no CSS principal: @import "tailwindcss";
4. Combinar classes no className: px-4 (espaço lateral), bg-indigo-600 (fundo), hover: (ao passar o mouse)

vite.config.ts

```
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/index.css

```
@import "tailwindcss";
```

src/Botao.tsx

```
<button
  className="rounded-lg bg-indigo-600 px-4 py-2
             text-white hover:bg-indigo-700"
>
  Salvar
</button>
```

FastAPI e Pydantic: uma API validada

Camada Controller

O que é

O FastAPI transforma uma função com decorador em uma rota HTTP. O Pydantic descreve o formato dos dados e recusa o que não bate, com mensagem clara.

Como usar

1. Instalar: pip install fastapi uvicorn
2. Descrever os dados com Pydantic
3. Criar a rota com o decorador
4. Rodar: uvicorn main:app --reload e abrir localhost:8000/docs

main.py

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class LivroEntrada(BaseModel):
    titulo: str = Field(min_length=1, max_length=100)
    paginas: int = Field(gt=0)

@app.post("/livros", status_code=201)
def criar(dados: LivroEntrada):
    return {"titulo": dados.titulo}
```

resposta 422 (paginas = 0)

```
{"detail": [{"loc": ["body", "paginas"],
  "msg": "Input should be greater than 0"}]}
```

JWT e bcrypt: login seguro

Camada Controller

O que é

O bcrypt transforma a senha em um hash com salt, e só o hash é guardado. O JWT é um token assinado que o servidor entrega no login e confere em cada pedido.

Como usar

1. Instalar: pip install bcrypt pyjwt
2. No cadastro, guardar apenas o hash da senha
3. No login, conferir a senha e emitir o token
4. Nas rotas protegidas, ler o cabeçalho Authorization: Bearer <token>

seguranca.py

```
import bcrypt, jwt
from datetime import datetime, timedelta, timezone

# cadastro: guarda só o hash
hash_ = bcrypt.hashpw(b"senha123", bcrypt.gensalt())

# login: confere a senha e gera o token
if bcrypt.checkpw(b"senha123", hash_):
    exp = datetime.now(timezone.utc) + timedelta(hours=2)
    token = jwt.encode({"sub": "42", "exp": exp},
                       SEGREDO, algorithm="HS256")

# rota protegida: valida e descobre quem pediu
dados = jwt.decode(token, SEGREDO, algorithms=["HS256"])
```

SQLAlchemy: classes viram tabelas

Camada Persistência

O que é

O SQLAlchemy é um ORM: cada classe vira uma tabela e cada objeto, uma linha. O repositório guarda as consultas, e a regra de negócio nem sabe que existe um banco.

Como usar

1. Instalar: pip install sqlalchemy "psycopg[binary]"
2. Declarar a classe que representa a tabela
3. Criar a engine apontando para o banco
4. Escrever o repositório e usá-lo no serviço

models.py

```
class Livro(Base):
    __tablename__ = "livro"
    id: Mapped[int] = mapped_column(primary_key=True)
    titulo: Mapped[str] = mapped_column(String(100))

engine =
create_engine("postgresql+psycopg://app:senha@db/oficina")
```

livro_repositorio.py

```
class LivroRepositorio:
    def __init__(self, sessao):
        self.sessao = sessao

    def salvar(self, titulo):
        livro = Livro(titulo=titulo)
        self.sessao.add(livro)
        self.sessao.commit()
        return livro
```

PostgreSQL e Alembic: banco e migrações

Camada Persistência

O que é

O PostgreSQL guarda os dados em tabelas, com chaves e restrições. O Alembic versiona o esquema: cada mudança vira um arquivo que pode ser aplicado ou desfeito.

Como usar

1. Instalar e iniciar: pip install alembic e alembic init migracoes
2. Apontar o migracoes/env.py para os modelos
3. Gerar: alembic revision --autogenerate -m "cria livro"
4. Aplicar: alembic upgrade head

migracoes/versions/001_cria_livro.py

```
def upgrade():
    op.create_table(
        "livro",
        sa.Column("id", sa.Integer, primary_key=True),
        sa.Column("titulo", sa.String(100)),
    )

def downgrade():
    op.drop_table("livro")
```

psql

```
oficina=# SELECT * FROM livro;
 id |    titulo
----+--------------
  1 | Dom Casmurro
```

Docker Compose: tudo com um comando

Ambiente

O que é

O Docker empacota cada serviço em um contêiner, descrito por um Dockerfile. O Compose descreve e liga vários contêineres em um único arquivo.

Como usar

1. Escrever o Dockerfile de cada serviço
2. Descrever os serviços no docker-compose.yml
3. Subir tudo: docker compose up -d --build e abrir localhost:5173
4. Limpar tudo: docker compose down -v

api/Dockerfile

```
FROM python:3.12-slim
COPY . /app
RUN pip install -r /app/requirements.txt
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

docker-compose.yml

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: senha
  api:
    build: ./api
    depends_on: [db]
    ports: ["8000:8000"]
  front:
    build: ./front
    ports: ["5173:80"]
```

Obrigado!

Código, diagramas e instruções: github.com/lucaslmchaves/sistema-moeda-estudantil